

ADVANCED WEB TECHNOLOGY

UNIT V

PHP Form Handling

We can create and use forms in PHP. To get form data, we need to use PHP superglobals \$_GET and \$_POST. The form request may be get or post. To retrieve data from get request, we need to use \$_GET, for post request \$_POST.

PHP Get Form

Get request is the default form request. The data passed through get request is visible on the URL browser so it is not secured. You can send limited amount of data through get request.

form1.html

```
<form action="welcome.php" method="get">
  Name: <input type="text" name="name"/>
  <input type="submit" value="visit"/>
</form>
```

welcome.php

```
<?php
$name=$_GET["name");//receiving name field value in $name variable
echo "Welcome, $name";
?>
```

PHP Post Form

Post request is widely used to submit form that have large amount of data such as file upload, image upload, login form, registration form etc. The data passed through post request is not visible on the URL browser so it is secured. You can send large amount of data through post request.

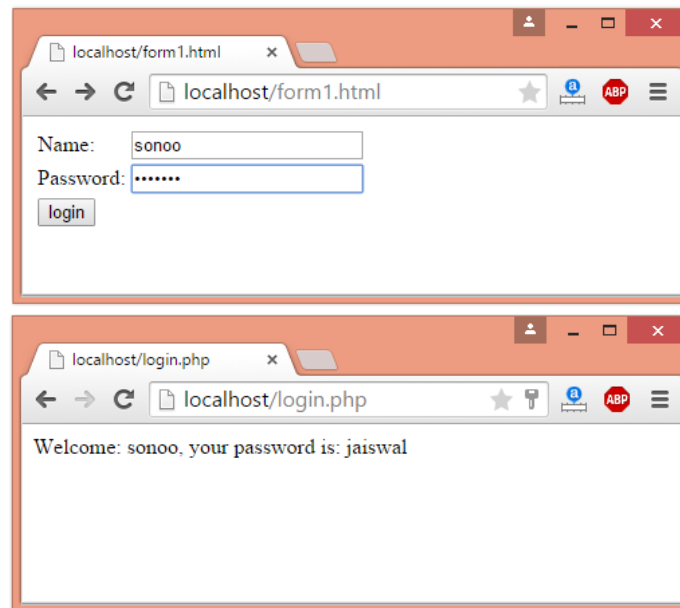
form1.html

```
<form action="login.php" method="post">
  <table>
    <tr><td>Name:</td><td> <input type="text" name="name"/></td></tr>
    <tr><td>Password:</td><td> <input type="password" name="password"/></td></tr>
  >
    <tr><td colspan="2"><input type="submit" value="login"/> </td></tr>
  </table>
</form>
```

login.php

```
<?php
$name=$_POST["name");//receiving name field value in $name variable
```

```
$password=$_POST["password");//receiving password field value in $password variable  
echo "Welcome: $name, your password is: $password";  
?>
```



Form Validation in PHP

An HTML form contains various input fields such as text box, checkbox, radio buttons, submit button, and checklist, etc. These input fields need to be validated, which ensures that the user has entered information in all the required fields and also validates that the information provided by the user is valid and correct. There is no guarantee that the information provided by the user is always correct. PHP validates the data at the server-side, which is submitted by HTML form. You need to validate a few things:

1. Empty String
2. Validate String
3. Validate Numbers
4. Validate Email
5. Validate URL
6. Input length

Empty String

The code below checks that the field is not empty. If the user leaves the required field empty, it will show an error message. Put these lines of code to validate the required field.

```
if (empty($_POST["name"])) {  
    $errMsg = "Error! You didn't enter the Name.";  
    echo $errMsg;  
} else {  
    $name = $_POST["name"];
```

```
}
```

Validate String

The code below checks that the field will contain only alphabets and whitespace, for example - name. If the name field does not receive valid input from the user, then it will show an error message:

```
$name = $_POST ["Name"];  
if (!preg_match ("/^[a-zA-z]*$/", $name) ) {  
    $ErrMsg = "Only alphabets and whitespace are allowed.";  
    echo $ErrMsg;  
} else {  
    echo $name;  
}
```

Validate Number

The below code validates that the field will only contain a numeric value. For example - Mobile no. If the *Mobile no* field does not receive numeric data from the user, the code will display an error message:

```
$mobilenos = $_POST ["Mobile_no"];  
if (!preg_match ("/^[0-9]*$/", $mobilenos) ){  
    $ErrMsg = "Only numeric value is allowed.";  
    echo $ErrMsg;  
} else {  
    echo $mobilenos;  
}
```

Validate Email

A valid email must contain @ and . symbols. PHP provides various methods to validate the email address. Here, we will use regular expressions to validate the email address. The below code validates the email address provided by the user through HTML form. If the field does not contain a valid email address, then the code will display an error message:

```
$email = $_POST ["Email"];  
$pattern = "^[_a-z0-9-]+(\\.[_a-z0-9-]+)*@[a-z0-9-]+(\\.[a-z0-9-]+)*(\\.[a-z]{2,3})$^";  
if (!preg_match ($pattern, $email) ){  
    $ErrMsg = "Email is not valid.";  
    echo $ErrMsg;  
} else {  
    echo "Your valid email address is: " . $email;  
}
```

Create and validate a Registration form

```
<!DOCTYPE html>
<html>
<head>
<style>
.error {color: #FF0001;}
</style>
</head>
<body>

<?php
// define variables to empty values
$nameErr = $emailErr = $mobilenErr = $genderErr = $websiteErr = $agreeErr = "";
$name = $email = $mobilen = $gender = $website = $agree = "";

//Input fields validation
if ($_SERVER["REQUEST_METHOD"] == "POST") {

//String Validation
    if (empty($_POST["name"])) {
        $nameErr = "Name is required";
    } else {
        $name = input_data($_POST["name"]);
        // check if name only contains letters and whitespace
        if (!preg_match("/^[a-zA-Z ]*$/", $name)) {
            $nameErr = "Only alphabets and white space are allowed";
        }
    }
}

//Email Validation
if (empty($_POST["email"])) {
    $emailErr = "Email is required";
} else {
    $email = input_data($_POST["email"]);
    // check that the e-mail address is well-formed
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $emailErr = "Invalid email format";
    }
}
}
```

```

//Number Validation
if (emptyempty($_POST["mobilen"])) {
    $mobilenErr = "Mobile no is required";
} else {
    $mobilen = input_data($_POST["mobilen"]);
    // check if mobile no is well-formed
    if (!preg_match ("/^[0-9]*$/", $mobilen) ) {
        $mobilenErr = "Only numeric value is allowed.";
    }
    //check mobile no length should not be less and greater than 10
    if (strlen ($mobilen) != 10) {
        $mobilenErr = "Mobile no must contain 10 digits.";
    }
}

//URL Validation
if (emptyempty($_POST["website"])) {
    $website = "";
} else {
    $website = input_data($_POST["website"]);
    // check if URL address syntax is valid
    if (!preg_match("/\b(?:(:https?|ftp):\\V|www\\.)([a-z0-9+&@#\\/%?~_|!:,.;]*[a-z0-9+&@#\\/%?~_|]/i",$website)) {
        $websiteErr = "Invalid URL";
    }
}

//Empty Field Validation
if (emptyempty ($_POST["gender"])) {
    $genderErr = "Gender is required";
} else {
    $gender = input_data($_POST["gender"]);
}

//Checkbox Validation
if (!isset($_POST['agree'])){
    $agreeErr = "Accept terms of services before submit.";
} else {
    $agree = input_data($_POST["agree"]);
}
}

```

```

function input_data($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
}
?>

```

<h2>Registration Form</h2>

* required field

<form method="post" action="<?php echo htmlspecialchars(\$_SERVER["PHP_SELF"]); ?>" >

Name:

<input type="text" name="name">

* <?php echo \$nameErr; ?>

E-mail:

<input type="text" name="email">

* <?php echo \$emailErr; ?>

Mobile No:

<input type="text" name="mobilenumber">

* <?php echo \$mobilenumberErr; ?>

Gender:

<input type="radio" name="gender" value="male"> Male

<input type="radio" name="gender" value="female"> Female

<input type="radio" name="gender" value="other"> Other

* <?php echo \$genderErr; ?>

Agree to Terms of Service:

<input type="checkbox" name="agree">

* <?php echo \$agreeErr; ?>

<input type="submit" name="submit" value="Submit">

</form>

<?php

if(isset(\$_POST['submit'])) {

```

        if($nameErr == "" && $emailErr == "" && $mobilenenoErr == "" && $genderErr == ""
        && $websiteErr == "" && $agreeErr == "") {
            echo "<h3 color = #FF0001> <b>You have sucessfully registered.</b> </h3>";
            echo "<h2>Your Input:</h2>";
            echo "Name: " . $name;
            echo "<br>";
            echo "Email: " . $email;
            echo "<br>";
            echo "Mobile No: " . $mobileneno;
            echo "<br>";
            echo "Gender: " . $gender;
        } else {
            echo "<h3> <b>You didn't filled up the form correctly.</b> </h3>";
        }
    }
?>
</body>
</html>

```

What is a Cookie?

A cookie is often used to identify a user. A cookie is a small file that the server embeds on the user's computer. Each time the same computer requests a page with a browser, it will send the cookie too. With PHP, you can both create and retrieve cookie values.

Create Cookies with PHP

A cookie is created with the `setcookie()` function.

Syntax:

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

Only the *name* parameter is required. All other parameters are optional.

PHP Create/Retrieve a Cookie

The following example creates a cookie named "user" with the value "John Doe". The cookie will expire after 30 days (86400 * 30). The "/" means that the cookie is available in entire website

```

<?php
$cookie_name = "user";
$cookie_value = "John Doe";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1 day
?>
<html>
<body>

```

```

<?php
if(!isset($_COOKIE[$cookie_name])) {
    echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
    echo "Cookie '" . $cookie_name . "' is set!<br>";
    echo "Value is: " . $_COOKIE[$cookie_name];
}
?>
</body>
</html>

```

Modify a Cookie Value

To modify a cookie, just set (again) the cookie using the setcookie() function:

Example:

```

<?php
$cookie_name = "user";
$cookie_value = "Alex Porter";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/");
?>
<html>
<body>
<?php
if(!isset($_COOKIE[$cookie_name])) {
    echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
    echo "Cookie '" . $cookie_name . "' is set!<br>";
    echo "Value is: " . $_COOKIE[$cookie_name];
}
?>
</body>
</html>

```

Delete a Cookie

To delete a cookie, use the setcookie() function with an expiration date in the past:

Example:

```

<?php
// set the expiration date to one hour ago
setcookie("user", "", time() - 3600);
?>
<html>
<body>
<?php

```



```
    echo "Cookie 'user' is deleted.";
?>
</body>
</html>
```

Check if Cookies are Enabled

The following example creates a small script that checks whether cookies are enabled. First, try to create a test cookie with the `setcookie()` function, then count the `$_COOKIE` array variable:

Example

```
<?php
setcookie("test_cookie", "test", time() + 3600, '/');
?>
<html>
<body>
<?php
if(count($_COOKIE) > 0) {
    echo "Cookies are enabled.";
} else {
    echo "Cookies are disabled.";
}
?>
</body>
</html>
```

What is a PHP Session?

When you work with an application, you open it, do some changes, and then you close it. This is much like a Session. The computer knows who you are. It knows when you start the application and when you end. But on the internet there is one problem: the web server does not know who you are or what you do, because the HTTP address doesn't maintain state.

Session variables solve this problem by storing user information to be used across multiple pages (e.g. username, favorite color, etc). By default, session variables last until the user closes the browser.

Start a PHP Session

A session is started with the `session_start()` function. Session variables are set with the PHP global variable: `$_SESSION`.

Now, let's create a new page called "demo_session1.php". In this page, we start a new PHP session and set some session variables:

Example:

```
<?php
// Start the session
session_start();
?>
<!DOCTYPE html>
<html>
<body>
<?php
// Set session variables
$_SESSION["favcolor"] = "green";
$_SESSION["favanimal"] = "cat";
echo "Session variables are set.";
?>
</body>
</html>
```

Get PHP Session Variable Values

Next, we create another page called "demo_session2.php". From this page, we will access the session information we set on the first page ("demo_session1.php"). Notice that session variables are not passed individually to each new page, instead they are retrieved from the session we open at the beginning of each page (session_start()). Also notice that all session variable values are stored in the global \$_SESSION variable:

Example:

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>
<?php
// Echo session variables that were set on previous page
echo "Favorite color is " . $_SESSION["favcolor"] . "<br>";
echo "Favorite animal is " . $_SESSION["favanimal"] . ".";
?>
</body>
</html>
```

Another way to show all the session variable values for a user session is to run the following code:

Example:

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>
<?php
print_r($_SESSION);
?>
</body>
</html>
```

Modify a PHP Session Variable

To change a session variable, just overwrite it:

Example:

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>
<?php
// to change a session variable, just overwrite it
$_SESSION["favcolor"] = "yellow";
print_r($_SESSION);
?>
</body>
</html>
```

Destroy a PHP Session

To remove all global session variables and destroy the session, use `session_unset()` and `session_destroy()`:

Example:

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
```

```
// remove all session variables
session_unset();
// destroy the session
session_destroy();
?>
</body>
</html>
```

What is MySQL?

- MySQL is a database system used on the web
- MySQL is a database system that runs on a server
- MySQL is ideal for both small and large applications
- MySQL is very fast, reliable, and easy to use
- MySQL uses standard SQL
- MySQL compiles on a number of platforms
- MySQL is free to download and use
- MySQL is developed, distributed, and supported by Oracle Corporation
- MySQL is named after co-founder Monty Widenius's daughter: My

PHP Connect to MySQL

PHP 5 and later can work with a MySQL database using:

- **MySQLi extension** (the "i" stands for improved)
- **PDO (PHP Data Objects)**

Earlier versions of PHP used the MySQL extension.

Should I Use MySQLi or PDO?

Both MySQLi and PDO have their advantages. PDO will work on 12 different database systems, whereas MySQLi will only work with MySQL databases. So, if you have to switch your project to use another database, PDO makes the process easy. You only have to change the connection string and a few queries. With MySQLi, you will need to rewrite the entire code - queries included. Both are object-oriented, but MySQLi also offers a procedural API. Both support Prepared Statements. Prepared Statements protect from SQL injection, and are very important for web application security.

MySQL Examples in Both MySQLi and PDO Syntax

In this, and in the following chapters we demonstrate three ways of working with PHP and MySQL:

- MySQLi (object-oriented)
- MySQLi (procedural)
- PDO

Open a Connection to MySQL

Before we can access data in the MySQL database, we need to be able to connect to the server:

Example (MySQLi Object-Oriented):

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
// Create connection
$conn = new mysqli($servername, $username, $password);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully";
?>
```

Example (MySQLi Procedural):

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
// Create connection
$conn = mysqli_connect($servername, $username, $password);
// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
?>
```

Example (PDO)

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
try {
    $conn = new PDO("mysql:host=$servername;dbname=myDB", $username,
    $password);
    // set the PDO error mode to exception
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connected successfully";
} catch(PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}
?>
```

Close the Connection

The connection will be closed automatically when the script ends. To close the connection before, use the following:

MySQLi Object-Oriented:

```
$conn->close();
```

MySQLi Procedural:

```
mysqli_close($conn);
```

PDO:

```
$conn = null;
```

Insert Data into MySQL Using MySQLi and PDO

After a database and a table have been created, we can start adding data in them. Here are some syntax rules to follow:

- The SQL query must be quoted in PHP
- String values inside the SQL query must be quoted
- Numeric values must not be quoted
- The word NULL must not be quoted

Example (MySQLi Object-oriented)

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";
// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
$sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('John', 'Doe', 'john@example.com')";
if ($conn->query($sql) === TRUE) {
    echo "New record created successfully";
} else {
    echo "Error: " . $sql . "<br>" . $conn->error;
}
$conn->close();
?>
```

Example (MySQLi Procedural)

```
<?php
$servername = "localhost";
```

```

$username = "username";
$password = "password";
$dbname = "myDB";
// Create connection
$conn = mysqli_connect($servername, $username, $password, $dbname);
// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
$sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('John', 'Doe', 'john@example.com')";
if (mysqli_query($conn, $sql)) {
    echo "New record created successfully";
} else {
    echo "Error: " . $sql . "<br>" . mysqli_error($conn);
}
mysqli_close($conn);
?>

```

Example (PDO)

```

<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDBPDO";
try {
    $conn = new PDO("mysql:host=$servername;dbname=$dbname", $username,
$password);
    // set the PDO error mode to exception
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('John', 'Doe', 'john@example.com')";
    // use exec() because no results are returned
    $conn->exec($sql);
    echo "New record created successfully";
} catch(PDOException $e) {
    echo $sql . "<br>" . $e->getMessage();
}

$conn = null;
?>

```